



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

Agosto- Diciembre 2025

CARRERA:

Ingeniería Informática

MATERIA:

Patrones de Diseño

TÍTULO ACTIVIDAD:

Examen Unidad 2

UNIDAD A EVALUAR:

Unidad 2

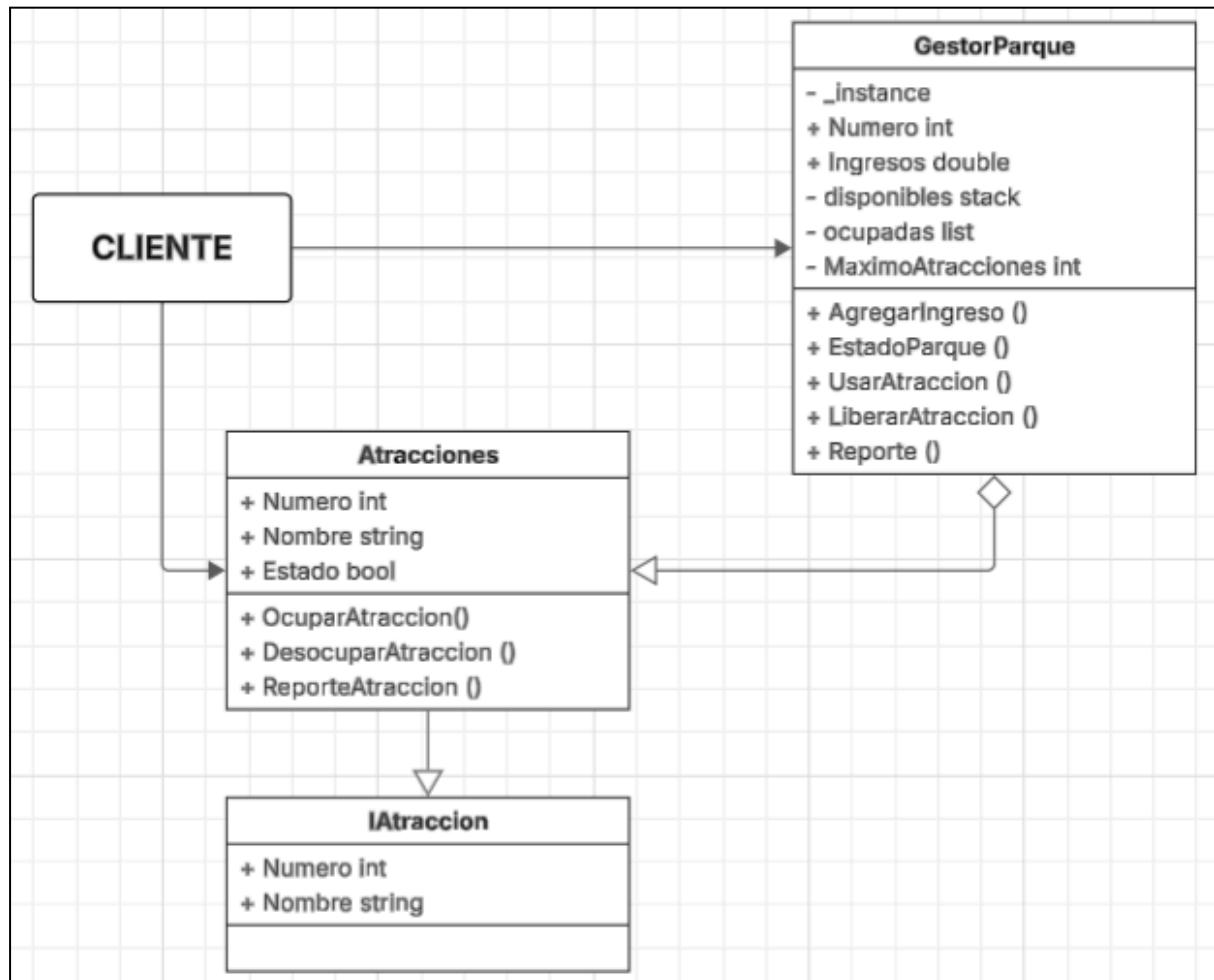
NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Miguel Angel Briones Hernandez 21212325

NOMBRE DEL MAESTRO (A):

Maribel Guerrero Luis

DIAGRAMA UML



CÓDIGO

IAtraccion

```
namespace ExamenU2
{
    public interface IAtraccion
    {
        int Numero { get; set; }
        string Nombre { get; set; }
    }
}
```

Atracciones

```
namespace ExamenU2
{
    public class Atracciones : IAtraccion
    {
        public int Numero { get; set; }
        public string Nombre { get; set; }
        public bool Estado { get; set; }

        public Atracciones(int numero, string nombre)
        {
            Numero = numero;
            Nombre = nombre;
            Estado = false;
        }

        public void OcuparAtraccion()
        {
            Estado = true;
            Console.WriteLine($"Atracción {Nombre} OCUPADA");
        }

        public void DesocuparAtraccion()
        {
            Estado = false;
            Console.WriteLine($"Atracción {Nombre} LIBRE");
        }

        public void ReporteAtraccion()
        {
            Console.WriteLine($"Atracción {Numero}. {Nombre} está: {Estado}");
        }
    }
}
```

GestorParque

```
namespace ExamenU2
{
    public class GestorParque
    {
        private static GestorParque _instance;
        private static readonly object _lock = new object();

        public string Nombre { get; private set; }
        public double Ingresos { get; private set; }

        private readonly Stack<Atracciones> disponibles = new Stack<Atracciones>();
        private readonly List<Atracciones> ocupadas = new List<Atracciones>();
        private readonly int MaximoAtracciones = 5;

        private GestorParque()
        {
            Nombre = "PARQUE EXAMEN UNIDAD 2";
            string[] nombres = { "Montaña Rusa", "Arcade", "Sillas voladoras", "Barco", "Rueda
de la fortuna" };

            for (int i = 0; i < MaximoAtracciones; i++)
            {
                var atrac = new Atracciones(i + 1, nombres[i]);
                disponibles.Push(atrac);
            }

            Console.WriteLine($"{Nombre} TIENE {MaximoAtracciones} ATRACCIONES\n");
        }

        public static GestorParque instance
        {
            get
            {
                if (_instance == null)
                {
                    lock (_lock)
                    {
                        if (_instance == null)
                        {
                            _instance = new GestorParque();
                        }
                    }
                }
                return _instance;
            }
        }
    }
}
```

```

}

public void AgregarIngreso(double cantidad)
{
    Ingresos += cantidad;
}

public void EstadoParque()
{
    Console.WriteLine($"El parque ha generado {Ingresos}$ en ingresos");
}

public Atracciones UsarAtraccion()
{
    if (disponibles.Count == 0)
    {
        Console.WriteLine("Atracciones Ocupadas");
        return null;
    }

    var atr = disponibles.Pop();
    atr.OcuparAtraccion();
    ocupadas.Add(atr);

    AgregarIngreso(50);

    return atr;
}

public void LiberarAtraccion()
{
    if (ocupadas.Count == 0)
    {
        Console.WriteLine("Todas las atracciones fueron liberadas");
        Console.ReadKey();
        Console.Clear();
        return;
    }

    var atr = ocupadas.Last(); // Libera la última ocupada
    atr.DesocuparAtraccion();
    ocupadas.Remove(atr);
    disponibles.Push(atr);
}

public void Reporte()
{

```

```

        Console.WriteLine("ESTADO DEL PARQUE\n");
        Console.WriteLine($"Hay {disponibles.Count} atracciones disponibles y
{ocupadas.Count} atracciones ocupadas:\n");

        foreach (var atr in disponibles.Concat(ocupadas).OrderBy(a => a.Numero))
            Console.WriteLine($"#{atr.Numero}. {atr.Nombre} - Ocupada: {atr.Estado}");
    }
}

```

Program

```

namespace ExamenU2
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int opc = 0;
            int opc2 = 0;

            var parque = GestorParque.instance;
            var comp = GestorParque.instance;

            parque.Reporte();
            Console.WriteLine("");
            parque.EstadoParque();

            do
            {
                Console.WriteLine("\nIndique que opción realizar: \r\n" +
                    "1. Ocupar atracción del parque\n" +
                    "2. Liberar atracción del parque\n" +
                    "3. Mostrar reporte del parque\n" +
                    "4. Mostrar ingresos generados\n");

                Console.Write("Selecciona una opción (1-4): ");
                opc = int.Parse(Console.ReadLine());
                Console.WriteLine("");

                switch (opc)
                {
                    case 1:
                        Console.Clear();
                        var atraccion = parque.UsarAtraccion();
                        parque.Reporte();

```

```
Console.WriteLine("");
parque.EstadoParque();
break;
```

case 2:

```
Console.Clear();
parque.LiberarAtraccion();
parque.Reporte();
Console.WriteLine("");
parque.EstadoParque();
break;
```

case 3:

```
Console.Clear();
parque.Reporte();
break;
```

case 4:

```
Console.Clear();
parque.EstadoParque();
break;
```

```
}
```

```
Console.WriteLine("");
Console.Write("¿Desea realizar otra acción? (1 = Si / 0 = No): ");
opc2 = int.Parse(Console.ReadLine());
Console.Clear();
```

```
} while (opc2 == 1);
```

```
}
```

```
}
```

```
}
```

CAPTURAS

```
C:\Users\Migue\source\repos  ×  +  v
PARQUE EXAMEN UNIDAD 2 TIENE 5 ATRACCIONES

ESTADO DEL PARQUE

Hay 5 atracciones disponibles y 0 atracciones ocupadas:

#1. Montaña Rusa - Ocupada: False
#2. Arcade - Ocupada: False
#3. Sillas voladoras - Ocupada: False
#4. Barco - Ocupada: False
#5. Rueda de la fortuna - Ocupada: False

El parque ha generado 0$ en ingresos

Indique que opción realizar:
1. Ocupar atracción del parque
2. Liberar atracción del parque
3. Mostrar reporte del parque
4. Mostrar ingresos generados

Selecciona una opción (1-4): |
```

```
Atracción Rueda de la fortuna OCUPADA
ESTADO DEL PARQUE

Hay 4 atracciones disponibles y 1 atracciones ocupadas:

#1. Montaña Rusa - Ocupada: False
#2. Arcade - Ocupada: False
#3. Sillas voladoras - Ocupada: False
#4. Barco - Ocupada: False
#5. Rueda de la fortuna - Ocupada: True

El parque ha generado 50$ en ingresos

¿Desea realizar otra acción? (1 = Si / 0 = No):
```


ESTADO DEL PARQUE

Hay 4 atracciones disponibles y 1 atracciones ocupadas:

- #1. Montaña Rusa - Ocupada: False
- #2. Arcade - Ocupada: False
- #3. Sillas voladoras - Ocupada: False
- #4. Barco - Ocupada: False
- #5. Rueda de la fortuna - Ocupada: True

¿Desea realizar otra acción? (1 = Si / 0 = No):

El parque ha generado 50\$ en ingresos

¿Desea realizar otra acción? (1 = Si / 0 = No):

C:\Users\Migue\source\repos

+

▼

Atracción Barco LIBRE
ESTADO DEL PARQUE

Hay 4 atracciones disponibles y 1 atracciones ocupadas:

- #1. Montaña Rusa - Ocupada: False
- #2. Arcade - Ocupada: False
- #3. Sillas voladoras - Ocupada: False
- #4. Barco - Ocupada: False
- #5. Rueda de la fortuna - Ocupada: True

El parque ha generado 100\$ en ingresos

¿Desea realizar otra acción? (1 = Si / 0 = No):

CONCLUSIÓN

El uso de patrones de diseño para la creación de aplicaciones ya sean de consola, móviles o webs permite que estas sean más eficientes, ligeras y óptimas sin sacrificar la robustez que pueda tener el proyecto.

En este ejemplo se pudo observar la implementación de dos patrones que permiten que el sistema consuma menos recursos otorgándole una funcionalidad más versátil. Para este caso práctico se hizo uso de los patrones singleton y object pool, por un lado singleton ayuda a gestionar el parque y su economía mientras que object pool las atracciones que existen dentro del mismo. Para hacer la combinación de ambos patrones se llegó a la conclusión de que si solo se habla de un parque en concreto entonces este debe ser una instancia única, por lo cual en sí mismo el parque debe de actuar como el singleton, sin embargo también se debe notar que el parque debe contener las atracciones por lo cual este debe ser la piscina de objetos que almacene las atracciones.

Lo realizado en el ejemplo fue crear una clase única (el singleton) que pudiese almacenar los objetos de una clase llamada atracciones, de modo que también fuese una piscina de objetos, en síntesis, si solo debe existir un parque que contenga todas las atracciones este puede ser tanto el pool de objetos como el singleton. Lo anterior permitió crear una aplicación donde solo existe un gestor de un solo parque que también contiene todas las atracciones que pueden ser ocupadas (salir del pool) y ser liberar (reingresar al pool).

En conclusión, los patrones de diseño permiten que un sistema sea más eficiente, útil e incluso seguro en algunos casos pues otorgan soluciones a problemas concretos. Sin embargo, hacer uso de estos de forma inteligente y combinada puede llevar a la creación de aplicaciones con mejores funcionalidades y eficiencia, pues en este caso no fue necesario crear un pool y la instancia del parque por separado, sino que estos pudieron unirse en uno solo.